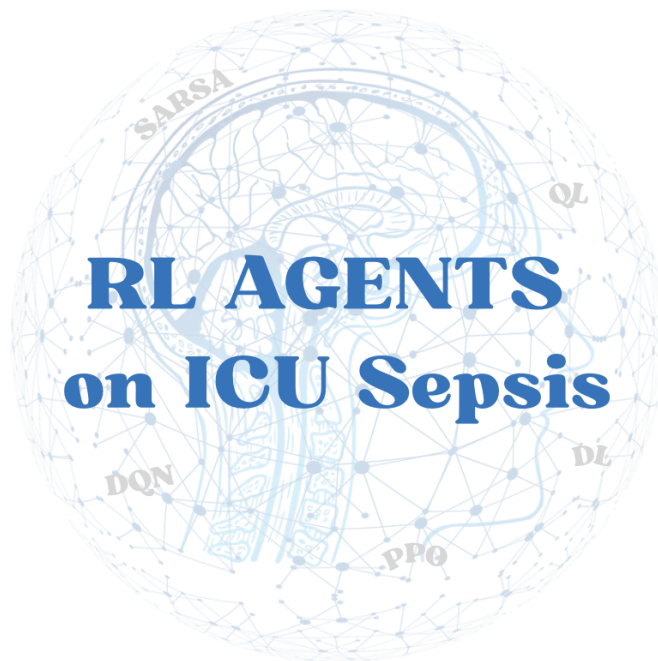


Reinforcement Learning Project
Master's in data science and advanced Analytics

NOVA Information Management School
Universidade Nova de Lisboa



Group B

Ana Margarida Macedo, 20250405

Mehmet Karaca, 20250344

Miguel Matos, 20221925

Luis Mendes, 20221949

Veronica Mendes, 20221945

Spring Semester 2025-2026

Table of Contents

1. Introduction	1
1.1. Environment & Problem Setup	1
2. Methodology	2
2.1 Configuration A: Discrete State Space	2
2.1.1 Dynamic programming (DP)	2
2.1.2 Q-Learning	3
2.1.3 SARSA	3
2.2 Configuration B: Continuous Observation Space	4
2.2.1 Deep Q-Network (DQN)	4
2.2.2 Proximal Policy Optimisation (PPO)	5
2.2.3 Hyperparameter Search	5
2.2.4 Best-Checkpoint Selection	5
3. Evaluation	5
3.1 Configuration A: Discrete State Space	6
3.1.1 Random Baseline	6
3.1.2 Dynamic Programming	6
3.1.3 Q-Learning	6
3.1.4 SARSA	7
3.2 Configuration B: Continuous Observation Space	8
3.2.1 Random Baseline	8
3.2.2 Training Performance	9
3.2.3 Clinical Wrapper Sensitivity	9
3.3 Comparison between configuration A and B	11
3.4 Creative Extension: Interpretability	12
4. Conclusion	15
References	16
Appendix	17

1. Introduction

Sepsis is one of the leading causes of mortality in intensive care units (ICUs) worldwide, accounting for millions of fatalities each year. It is a complex, life-threatening condition triggered by the body's overwhelming response to an infection. Managing a septic patient requires continuous, high-stakes medical decisions centered around balancing two primary interventions: administering vasopressor medications to stabilize blood pressure and providing intravenous (IV) fluids to maintain systemic circulation. This clinical management is a delicate balancing act; administering too little intervention allows the patient to rapidly deteriorate, while administering too much can cause severe secondary harm and treatment-induced toxicity. Historically, these continuous, detailed decisions have been made by ICU clinicians relying heavily on subjective personal experience and generic clinical guidelines.

The inherent limitations of static clinical guidelines become apparent when confronted with the vast complexity of sepsis progression, which features hundreds of evolving patient states, dozens of potential intervention actions, highly delayed physiological outcomes, and extreme individual patient variability. This multi-dimensional, sequential decision-making problem under uncertainty makes sepsis treatment a natural candidate for Reinforcement Learning (RL). Unlike traditional statistical models, a well-trained RL agent can learn dynamically optimized treatment policies directly from historical patient data. By modeling the ICU environment, the agent can map diverse clinical trajectories and uncover complex, personalized treatment strategies designed to maximize terminal survival rates while simultaneously minimizing unnecessary, high-intensity interventions.

In this project, we address this sequential optimization challenge by developing and evaluating reinforcement learning agents on the ICU-Sepsis-v2 benchmark, an environment built directly from the real-world MIMIC-III clinical database. Each training and evaluation episode simulates an individual ICU patient trajectory, where the agent observes the patient's clinical variables at each step and selects an intervention combining vasopressor and IV fluid dosage levels. The episode terminates when the patient either survives (yielding a terminal reward of +1.0) or dies (yielding a reward of 0); the primary objective of our agents is to maximize this survival rate while simultaneously minimizing unnecessary treatment intensity.

1.1. Environment & Problem Setup

ICU-Sepsis-v2 is a discrete-time MDP benchmark constructed from the MIMIC-III clinical database (Johnson et al., 2016; Komorowski et al., 2018). Each episode simulates one ICU patient trajectory: at every step the agent observes the patient's current state, selects a treatment action, receives a reward signal, and transitions to the next state until the episode terminates in either survival or death. The action space is shared across both configurations, with 25 possible discrete actions, representing every combination of five vasopressor dose levels and five IV fluid dose levels (none / low / medium / high / very high). Action 0 prescribes no treatment; action 12 prescribes medium doses of both interventions; action 24 prescribes maximum doses of both. To discourage unnecessary over-treatment, a per-step intensity penalty $\lambda = 0.02$ is subtracted from the reward at each timestep, so the effective reward is +1

($0.02 \times \text{intensity}$) at survival, and ($0.02 \times \text{accumulated intensity}$) at death. This means episode return is always slightly below survival rate, with the gap reflecting total treatment cost.

The two configurations differ only in how the patient state is represented. Configuration A, encodes the patient's condition as a single discrete integer in the range $[0, 715]$, derived by discretising the raw physiological measurements into 716 clinically defined categories. Because the state space is finite and the full MDP, transition probability tensor $P(s'|s,a)$ and reward tensor $R(s,a,s')$ is accessible from the environment internals, model-based planning methods such as Dynamic Programming can be applied directly. A Q-table for this configuration requires only 17,900 entries ($716 \text{ states} \times 25 \text{ actions}$), making tabular model-free methods entirely tractable. Configuration B, replaces the discrete index with the raw 47 dimensional physiological feature vector used in the original AI Clinician study: SOFA score, heart rate, mean arterial pressure, lactate, creatinine, and 42 additional clinical variables. The observation space is now continuous, the agent never encounters the exact same state twice, rendering a finite Q table computationally infeasible. Three clinical failure wrappers additionally stress-test the agents: 'EpisodicNoisyObsEnv' applies Gaussian noise to all observations for randomly selected episodes (15% of episodes), 'EpisodicMissingObsEnv' zeroes out four lab features for the duration of an episode (15% of episodes) and 'AcuteEventEnv' introduces a 1% per-step probability of sudden irreversible patient death independent of any treatment decision.

2. Methodology

For every selected algorithm, a comprehensive optimisation process was conducted to identify the hyperparameter configuration, spanning learning rates, exploration schedules, and network architectures, that yields stable treatment policies. We then perform convergence analysis and learning curve evaluation across all models, tracking metrics such as survival rate per episode and training velocity. This allows us to compare how effectively tabular and deep function approximation paradigms transition from exploration to exploitation, ensuring that algorithmic comparisons are both statistically grounded and clinically interpretable.

2.1 Configuration A: Discrete State Space

We selected Dynamic Programming (DP), Q-Learning, and SARSA as our three core algorithms. This selection is highly deliberate: it spans fundamentally different RL paradigms within the tabular domain. By incorporating these three methods, we can directly contrast model-based planning (DP) against model-free learning (Sutton & Barto, 2018), while simultaneously evaluating the empirical trade-offs between off-policy updates (Q-Learning) and on-policy updates (SARSA) in a safety-critical clinical environment.

2.1.1 Dynamic programming (DP)

Dynamic Programming (DP) is a model-based method that computes the optimal policy directly from the known transition matrix $P(s'|s,a)$ and reward matrix $R(s,a,s')$, without requiring environment interaction during training. This makes it well-suited to ICU-Sepsis, where full MDP dynamics are available and unnecessary exploration raises patient safety concerns.

Two algorithms were implemented: Policy Iteration, which alternates between full policy evaluation (Bellman expectation equation) and greedy policy improvement until the policy stabilises and Value Iteration, which combines both steps into a single Bellman optimality update per iteration. Both used a convergence threshold of 10^{-6} and were tested across $\gamma \in \{0.90, 0.95, 0.99, 1.0\}$, with $\gamma = 1.0$ adopted as the final setting following the ICU-Sepsis paper convention.

Bellman expectation equation (Policy Iteration): $V'(s) \leftarrow \sum P(s', r|s, \alpha) [r + \gamma V(s')]$

Bellman optimality equation (Value Iteration): $V'(s) \leftarrow \max_a \sum P(s', r|s, \alpha) [r + \gamma V(s')]$

2.1.2 Q-Learning

Q-Learning is a model-free, off-policy temporal difference algorithm that learns the optimal action-value function $Q(s, a)$ directly from environmental interactions, without requiring access to transition probabilities or reward matrices. At each step, the agent observes a state, selects an action, receives a reward, and updates its Q-table according to the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

The off-policy nature of Q-Learning means the update target always bootstraps from the greedy maximum over next-state values, regardless of which action the agent actually takes next. This allows the algorithm to learn the optimal policy while simultaneously following an exploratory behaviour policy, making it particularly well suited to the sepsis management task where safe exploration and optimal exploitation must be carefully balanced.

Exploration is governed by an ϵ -greedy strategy, starting with a high ϵ , that will linearly decay over a configurable number of steps toward a minimum value ϵ end, progressively shifting the agent toward greedy action selection as the Q-table matures.

The key hyperparameters of Q-Learning behaviour are the learning rate α , which controls how aggressively new information overrides existing estimates; the discount factor γ , which balances the weight of immediate versus future rewards; the decay schedule ϵ decay, which determines the speed of the exploration-to-exploitation transition; and ϵ end, which sets the floor on residual exploration maintained throughout training. To identify the optimal combination of these parameters, a systematic hyperparameter search was conducted, selecting the configuration that maximised patient survival rate.

2.1.3 SARSA

SARSA (State Action Reward State Action), is an on-policy Temporal Difference (TD) learning algorithm used to estimate the action-value function $Q(s, a)$ for the discrete state-space configuration on the ICU-Sepsis. Unlike off-policy algorithms like Q-learning, which optimize purely for the theoretical optimal path assuming perfect exploitation, SARSA updates its action-value estimates using the actual action selected by the current behavioral policy (ϵ -greedy). The updates follow the classic TD(0) update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

Where:

- s, a are the current state and chosen treatment action.
- r is the clinical reward feedback from the environment.
- s', a' are the subsequent state and the actual next action selected by the agent's ϵ -greedy exploratory schedule.

Because SARSA updates its memory based on the actual next action it chooses, it is naturally safer during training than algorithms like Q-Learning. In a high-stakes ICU environment where dangerous medical decisions (such as mismatched drug dosages) can severely harm a patient, SARSA avoids risky behaviors by accounting for its own random mistakes while exploring. To find the best setup for this agent, an automated Optuna search maximized the patient's survival rewards by evaluating the parameters detailed in Table A1 (Appendix A). This search tuned the learning rate (α) and its decay schedule to control how fast the agent learns from new patient data, as well as the discount factor (γ) to balance immediate stabilization with long-term recovery. It also optimized the exploration schedule (ϵ -greedy) to safely guide the agent from early trial-and-error treatment to final, confident medical decisions. Crucially, the search tested different initial memory settings (Q-table initialization (Q_{init})) to see if changing the agent's starting expectations would improve how it explores different treatment paths. Looking at these initial settings broadly allowed the optimization process to analyze whether pre-existing assumptions or complete neutrality helps an algorithm uncover safer medical strategies.

2.2 Configuration B: Continuous Observation Space

Because the observation space is continuous, tabular methods are no longer applicable and function approximation via neural networks becomes necessary. We selected Deep Q-Network (DQN) and Proximal Policy Optimisation (PPO) as our two algorithms. This selection is mathematically deliberate, as it directly contrasts a value-based, off-policy approach (DQN) against a policy-gradient, on-policy framework (PPO). Both are implemented using Stable-Baselines3 with custom MLP policies and trained with gamma = 1.0, matching the environment convention of undiscounted episodic returns.

2.2.1 Deep Q-Network (DQN)

DQN is an off-policy, value-based method (Mnih et al., 2015). It approximates the action-value function $Q(s, a)$ with a neural network and trains on mini-batches drawn from an experience replay buffer. Two mechanisms address the instability that arises when neural networks are used as function approximators: (i) the replay buffer breaks temporal correlations in the training data, and (ii) a periodically synchronised target network provides stable regression targets. DQN is a natural extension of Q-Learning to the continuous observation setting and is well-suited to the discrete 25-action space of ICU-Sepsis-v2.

The network architecture uses two fully-connected layers of 64 units with ReLU activations (Kaiming uniform initialisation, the PyTorch default and the appropriate choice for ReLU networks). Final hyperparameters are listed in Table A2 (Appendix A).

2.2.2 Proximal Policy Optimisation (PPO)

PPO is an on-policy, policy-gradient method. Rather than learning a value function and deriving a greedy policy, PPO directly parameterises the policy $\pi(a|s)$ and optimises it via a clipped surrogate objective (Schulman et al., 2017). The clipping prevents the policy from moving too far from the current iterate in a single update, which is the key stability improvement over vanilla policy gradient methods. PPO is the standard deep RL baseline for environments with discrete actions and moderate episode lengths, and its on-policy nature provides a useful contrast to DQN's replay-based training.

The network uses two layers of 128 units with ReLU activations. PPO's on-policy nature requires larger rollouts to reduce gradient variance, and the actor-critic architecture benefits from a wider network than DQN. Final hyperparameters are listed in Table A3 (Appendix A).

2.2.3 Hyperparameter Search

A two-stage search was conducted to identify the best configuration for each algorithm. First, an exhaustive grid search evaluated 486 DQN and 648 PPO configurations, each trained for 100,000 steps and assessed on 200 evaluation episodes. Second, Bayesian optimization with Optuna's TPE sampler ran 20 additional trials per algorithm at the full training budget, refining dimensions not covered by the grid (notably *target_update_interval* for DQN). The grid-best configuration was seeded as trial 0 in both studies so improvements over the grid baseline are directly measurable.

2.2.4 Best-Checkpoint Selection

Training curves for DQN in deep RL settings frequently exhibit non-monotonic behaviour: the agent may reach a strong policy mid-training that subsequently degrades due to replay buffer shifts or exploration noise. To guard against saving an inferior final step model, we implemented a *CheckpointCallback* that evaluates the current policy over 100 deterministic episodes every 10,000 steps and records the survival rate at each checkpoint. At the end of training, the best performing snapshot is restored before the model is saved. The full training curve and checkpoint evaluation results are presented in Section 3.2. Furthermore, to evaluate how the models adapt to different starting conditions and gradient flows, we also conducted a comparative analysis of alternative weight initialization techniques.

3. Evaluation

This section presents the training and evaluation results for all algorithms across both configurations. For each method, we report the training curve, convergence behaviour, and final survival rate against the respective random baseline. Section 3.3 consolidates all results into a

unified comparison table and discusses what the transition from Configuration A to B reveals about the scalability of different RL paradigms.

3.1 Configuration A: Discrete State Space

3.1.1 Random Baseline

A uniformly random policy evaluated over 1,000 episodes (seed = 42) achieves a survival rate of 74.3% in the full clinical environment. This serves as the reference floor; all trained agents are measured against it.

3.1.2 Dynamic Programming

Both Policy Iteration and Value Iteration successfully converged to the optimal policy for the ICU-Sepsis MDP. Policy Iteration converged in 4 iterations, while Value Iteration required 135 iterations, which is expected, since Policy Iteration converges in fewer outer iterations and Value Iteration performs partial updates and requires more sweeps to propagate values accurately across all states (see Fig B1, Appendix B).

Despite the difference in convergence speed, both algorithms produced identical final policies. The evaluation results over 1,000 episodes with $\gamma = 1.0$ are 83.2% of survival rate, which represents a substantial improvement over a random treatment policy (74.3%). This demonstrates that even a relatively simple model-based approach can learn clinically meaningful treatment strategies when the full MDP dynamics are available.

A key limitation of DP in this context is its dependence on a known model. In real clinical practice, the true transition probabilities are never fully known, they are estimated from historical data. This means DP results represent an upper bound on what model-based methods can achieve given perfect knowledge, and model-free approaches explored in the following sections must close this gap through experience.

3.1.3 Q-Learning

Q-Learning was trained for 30,000 episodes using the best configuration from a grid search over 27 hyperparameter combinations ($\alpha \in \{0.05, 0.1, 0.2\}$, $\epsilon_{decay} \in \{5000, 10000, 25000\}$, $\epsilon_{end} \in \{0.01, 0.05, 0.10\}$, with $\gamma = 1.0$ fixed). The best configuration ($\alpha = 0.1$, $\epsilon_{decay} = 25000$, $\epsilon_{end} = 0.1$) is used for the full training run.

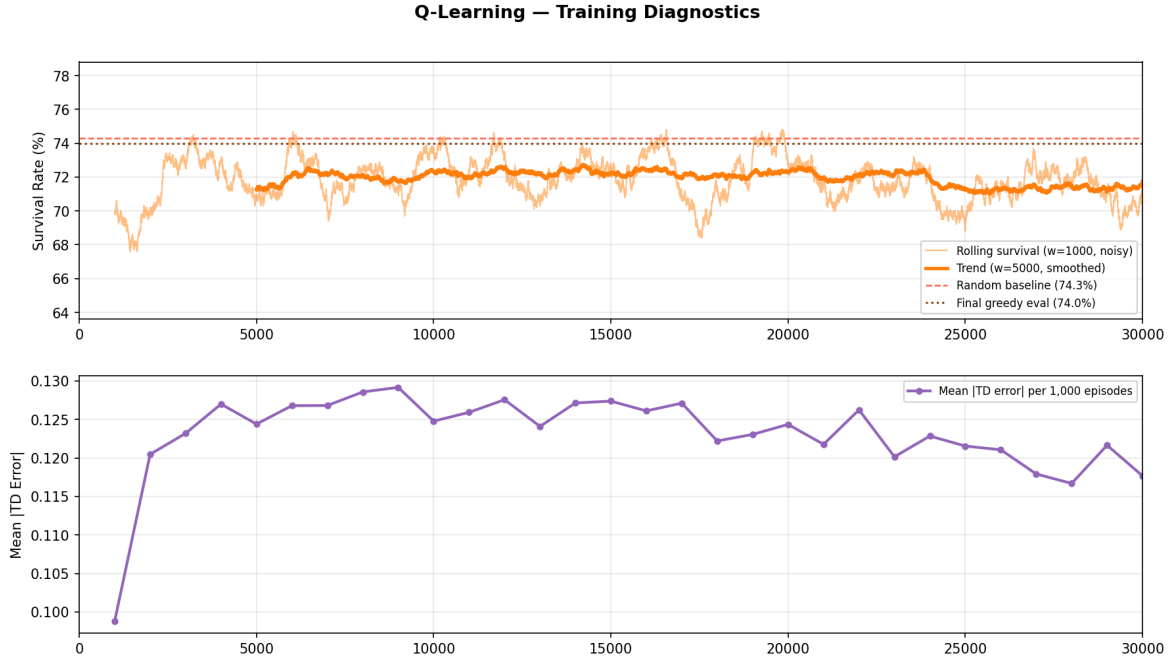


Figure 1: Training diagnostic for Q-Learning: rolling survival rate (top) and mean absolute TD error (bottom).

Driven by initial aggressive exploration (see Fig B2, Appendix B), the mean absolute TD error peaks near episode 9,000 as zero initialized value estimates clash with newly discovered terminal rewards, before steadily declining as the policy transitions to exploitation. Consequently, the final greedy evaluation yields a survival rate of 74.0% (−0.3 pp vs. the random baseline, with a mean return of 0.651). The stagnation of the survival rate improvement demonstrates that the exploration-exploitation schedule was insufficient to reliably propagate the environment's delayed, sparse binary rewards backward through the 716 discrete states within this localized training budget.

3.1.4 SARSA

SARSA was implemented as the tabular, on-policy counterpart to off-policy Q-Learning in Configuration A. The optimal hyperparameter configuration identified a learning rate of $\alpha = 0.05$, a discount factor of $\gamma = 0.95$, a uniform Q-table initialization at 0.0, and an exploration schedule defined by $\epsilon_{\text{start}} = 0.5$, $\epsilon_{\text{end}} = 0.2$, and linear epsilon decay of 500 steps (see Fig B2, Appendix B).

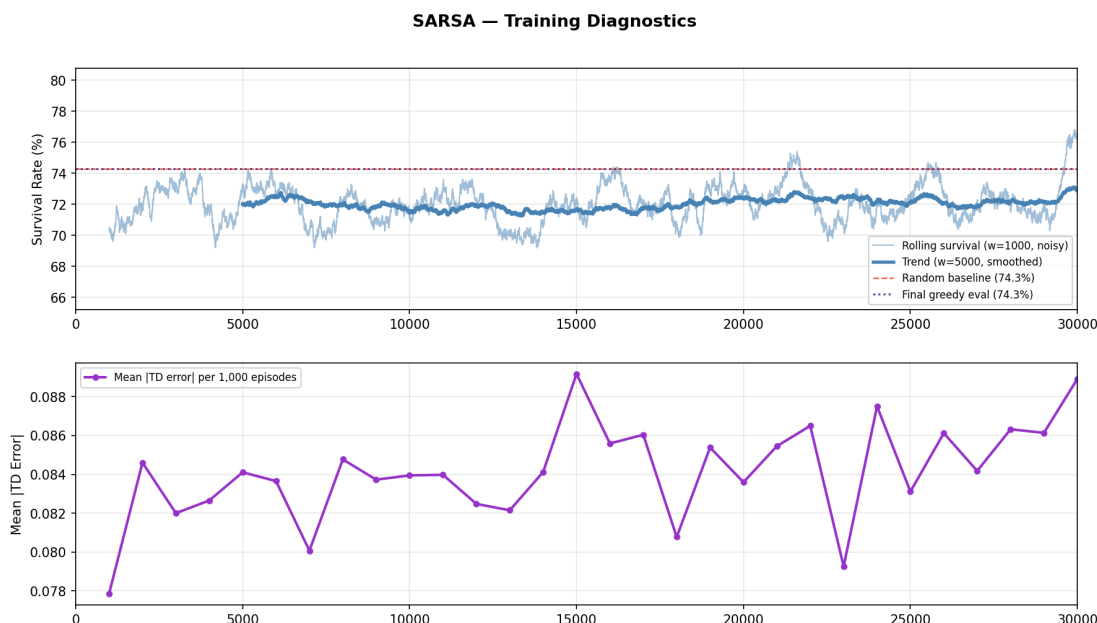


Figure 2: Tabular SARSA Training Diagnostics: 30,000-episode rolling survival rate (top) and mean absolute TD error (bottom).

The SARSA training diagnostics reveal a highly volatile learning trajectory heavily governed by its on-policy update rule and a restrictive exploration schedule. Unlike off-policy alternatives, SARSA updates its value functions based on the actions actually executed by its exploratory policy, meaning it explicitly accounts for its own random mistakes during training. This operational characteristic was severely constrained by an exceptionally abrupt initial exploration decay window (epsilon start = 0.5, epsilon end = 0.2, over approximately 50 episodes), which severely curtailed early trial-and-error discovery. Furthermore, because a high exploration floor (epsilon = 0.2) was maintained across the remaining 30,000 episodes, the agent was subjected to continuous random actions; this persistent behavioral variance is visually confirmed by the highly erratic, oscillating Mean Absolute TD Error curve that fails to establish a clean downward convergence profile. Faced by continuous step-level noise, the model encountered immediate intermediate step punishments from the treatment intensity penalty long before the sparse, delayed terminal survival rewards (+1.0) could reliably propagate backward to early clinical states. Consequently, the agent developed a highly conservative strategy that collapsed into systematic inaction to mitigate immediate penalty accumulation, causing the final greedy evaluation to plateau at 74.3%, directly matching the performance ceiling of the random baseline.

3.2 Configuration B: Continuous Observation Space

3.2.1 Random Baseline

A uniformly random policy evaluated over 1,000 episodes (seed = 42) achieves a survival rate of 68.3% in the full clinical environment. This serves as the reference floor, all trained agents are measured against it.

3.2.2 Training Performance

Both DQN and PPO were trained for 100,000 timesteps on the full clinical environment, demonstrating severe optimization instability and cyclical performance rather than an idealized, monotonic upward learning trajectory. Visually, the rolling survival curves show DQN fluctuating heavily, a pattern closely tied to its linear transition from full exploration to 90% greedy action selection over the first 25,000 steps (see Fig B3, Appendix B), while PPO undergoes a massive performance drop mid-training (see Fig B4, Appendix B). This extensive volatility highlights the immense difficulty deep RL agents face when propagating sparse, delayed rewards through heavily perturbed clinical environments. The quantitative metrics from the results table further confirm these training diagnostics. Utilizing the checkpoint recovery mechanism to isolate viable clinical policies before late-stage decay occurs, DQN achieved its best intermediate survival rate of 68% at step 40,000, while PPO reached its peak at step 70,000 with a 71% survival rate (see Table 1). However, when subjected to the final clinical evaluation across 1,000 independent validation episodes, distinct policy degradation is clear: DQN dropped to 66.2% and PPO fell to 67.4%. Consequently, both finalized models ultimately underperformed relative to the random baseline's 68.3% survival rate.

Table 1: Survival Rates for DQN and PPO.

ALGORITHM	BEST CHECKPOINT STEP	BEST CHECKPOINT SURVIVAL	FINAL CLINICAL EVAL (1,000 EP)
DQN	40.000	68%	66.2%
PPO	70.000	71%	67.4%
RANDOM BASELINE			68.3%

To evaluate optimization sensitivity under these specific hyperparameter settings, different weight initialization methods were systematically tested, confirming that the default configuration for each algorithm yielded the absolute best results and was chosen for the final implementation. Specifically, Kaiming Uniform initialization for DQN (architecturally designed for ReLU activations) and Orthogonal initialization for PPO (the Stable-Baselines3 default for policy gradient methods) were confirmed as optimal. Swapping these initializations significantly degraded performance, causing a 4 to 5 percentage point drop in clean-environment survival rates for each algorithm. This distinct performance delta validates that the standard default initializations are highly appropriate and well-motivated for stabilizing gradient flow and maximizing policy efficacy in this continuous clinical task (see Fig B5, Appendix B).

3.2.3 Clinical Wrapper Sensitivity

To isolate the contribution of each failure mode, both agents were evaluated under five wrapper configurations (500 episodes each), as illustrated in Figure 3:

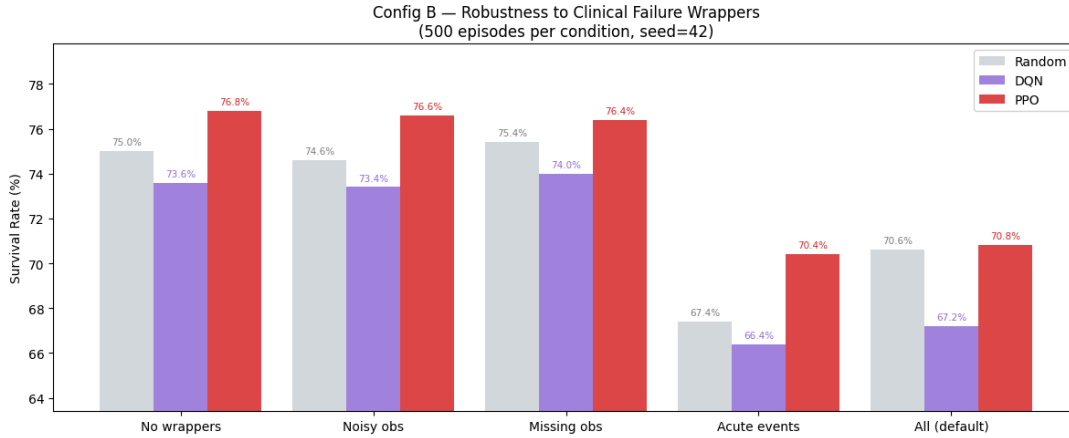


Figure 3: Survival rate of DQN, PPO, and a random policy under each wrapper condition.

Two findings stand out. First, the ‘AcuteEventEnv’ wrapper is the dominant failure mode: removing all wrappers raises survival by 6-7 pp for both agents, whereas adding only the acute events wrapper nearly replicates the full-clinical result. With a 1% per-step sudden-death probability and episodes averaging 10 steps, approximately 9% of patients die regardless of any treatment decision, this ceiling cannot be overcome by better learning. Second, noisy and missing observations have negligible impact (<1.5 pp loss for either agent), confirming that 47-dimensional physiological observations carry sufficient redundancy for the neural networks to remain robust to partial data corruption. PPO consistently outperforms DQN across all conditions, with the largest gap appearing under acute events (+4.0 pp). This advantage likely reflects PPO’s on-policy training distribution: rollout data is collected under the same noisy test conditions the policy will face at test time, whereas DQN’s replay buffer retains experience from earlier, potentially less noisy training phases.

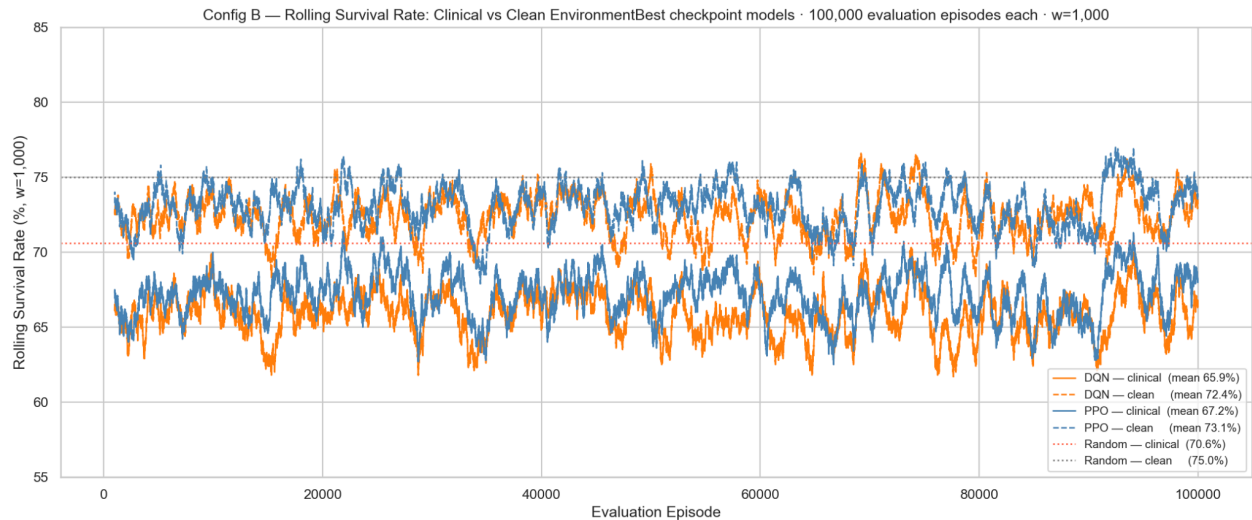


Figure 4: Survival Rate Clean vs Clinical Environment.

The multi-environment evaluation on Figure 4 highlights a stark contrast in policy viability when confronting clinical operational noise. In the clean environment, both models are capable of performing better than the random baseline (75.0%) with some frequency, achieving periodic performance surges that break above the benchmark line. However, introducing real-world degradation in the clinical environment severely dampens this efficacy, making it significantly more difficult for either agent to establish or maintain a meaningful positive difference against the corresponding random baseline (70.6%). Crucially, the models struggle to achieve true mathematical convergence in both settings; rather than demonstrating a stable upward trajectory, they remain confined within a highly volatile, constant survival rate range across the entire 100,000 evaluation episodes.

3.3 Comparison between configuration A and B

The table below consolidates all final results using the fixed evaluation protocol (1,000 episodes, seed = 42) for both configurations:

Table 2: Comparative Algorithm Performance across Configuration A (Discrete) and B (Clinical and Clean).

	ALGORITHM	SURVIVAL RATE	VARIATION	MEAN RETURN
CONFIGURATION A Discrete (clean) Environment	Random	74.3%		0.640
	DP	83.2%	+8.9pp	0.791
	Q-learning	74.0%	-0.3pp	0.651
	Sarsa	74.3%	0.0pp	0.719
CONFIGURATION B Clinical (all wrappers) Environment	Random	68.3%		0.586
	DQN	67.2%	-1.1pp	0.586
	PPO	70.8%	+2.5pp	0.635
CONFIGURATION B Clean Environment	Random	75.0%		0.640
	DQN	73.6%	-1.4pp	0.644
	PPO	76.8%	+1.8pp	0.698

Dynamic Programming is the only algorithm that reliably exceeds random (+8.9 pp), enabled by direct access to the full MDP transition model. All four model-free methods, both tabular (Config A) and deep (Config B), remain near or slightly below their respective random baselines. The binding constraint is identical across both configurations: sparse binary terminal rewards make credit assignment over ~10-step episodes unreliable at the training budgets used. Deep RL's advantage over tabular methods in Config B is not a higher survival rate but rather its ability to operate at all in a continuous 47-dimensional observation space where a Q-table is computationally infeasible. This scalability to continuous observations is the critical requirement for real clinical deployment, where patient states are always represented as raw physiological measurements and a discrete state index is never available.

3.4 Creative Extension: Interpretability

Evaluating reinforcement learning agents solely on quantitative metrics can mask dangerous or illogical medical policies. Interpretability analysis was chosen to evaluate whether the learned behaviors align with established intensive care protocols and safety considerations. It was expected that clinically optimized policies would demonstrate an empirical adaptation to the step-intensity penalty by favoring parsimonious, structured interventions over aggressive, random dosing. Instead of arbitrary behaviors, it was found that the highest-performing agents developed highly structured strategies with a clear preference for low-intensity interventions, whereas weaker tabular methods suffered from structural training limitations.

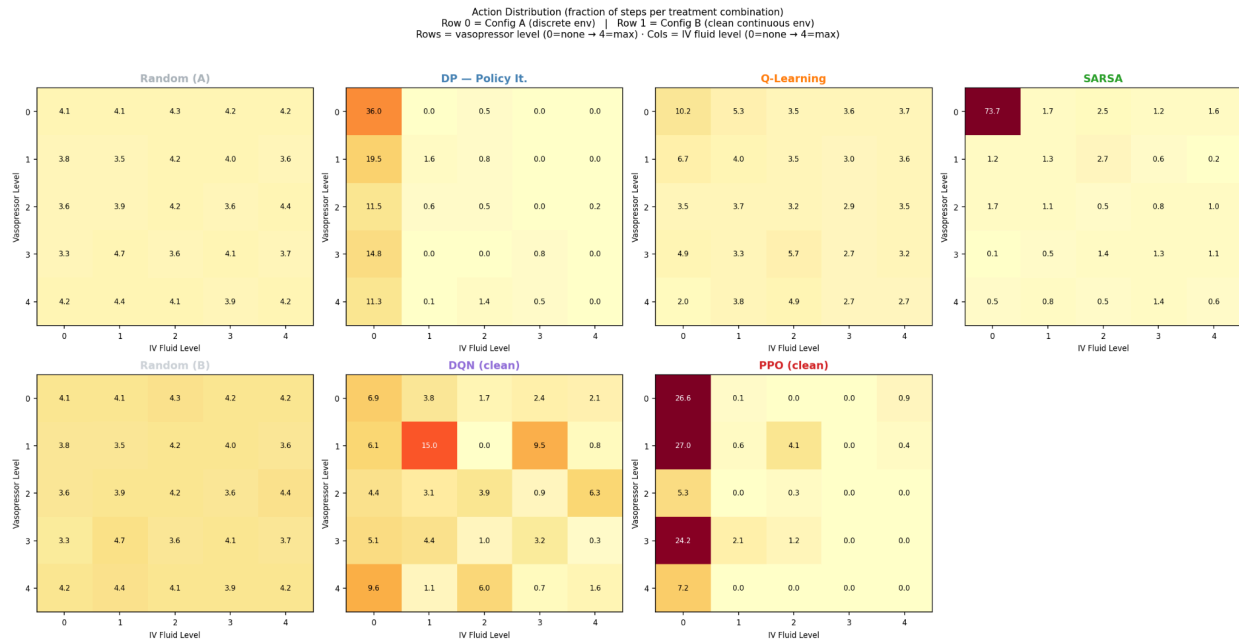


Figure 5: Actions Distribution Analysis.

The action distributions provide insight into how each agent approaches patient management using combinations of intravenous (IV) fluids and vasopressor dosages. From a clinical perspective, IV fluids are administered to increase circulating blood volume and improve tissue perfusion, while vasopressors raise blood pressure when fluids alone are insufficient.

The Random A and Random B policies exhibited an almost uniform distribution across all 25 possible actions, indicating a flat prior with no coherent treatment strategy. They achieve their survival baselines simply because a large portion of the patient cohort survives regardless of the interventions applied.

The Dynamic Programming (DP) policy demonstrated the most concentrated action distribution, with more than 90% of the selected actions involving no IV fluid administration while varying the vasopressor dosage. The most frequently selected action corresponded to no IV fluids and no vasopressors, accounting for 36.0% of all decisions. The consistency of this behaviour makes the DP policy highly structured and interpretable.

The Q-Learning policy exhibited a more dispersed, diffuse action pattern across treatment combinations. While it shows a modest preference for low-intensity actions, the policy is less straightforward to interpret. This dispersion does not reflect an intentional treatment diversity strategy; rather, it indicates that the tabular state-action space is too sparsely trained for the agent to commit to a clearly defined clinical pattern.

The SARSA policy showed the strongest concentration on a single action, selecting the combination of no IV fluids and no vasopressors in approximately 73.7% of all decisions. Inferring that the best option is to do nothing.

The Deep Q-Network (DQN) policy displayed a balanced distribution across multiple treatment combinations, utilizing several levels of both IV fluids and vasopressors from the continuous observation vector. This suggests a more flexible treatment approach that adapts to a wider range of continuous patient physiological parameters.

The Proximal Policy Optimization (PPO) policy learned a structured strategy characterized by a strong preference for actions involving low IV fluid administration combined with varying vasopressor levels. PPO distributed its decisions across several vasopressor intensities rather than concentrating on a single action, indicating a more nuanced treatment strategy while maintaining a clear policy structure.

Overall, the structural comparison between DP and PPO is highly revealing. Despite utilizing completely different learning mechanisms, DP derives its strategy analytically from the full MDP transition matrix and PPO learning implicitly from continuous physiological vectors, both agents prefer specific, concentrated treatment combinations over uniform random allocation (both recommend low IV Fluid level and change only the vasopressor level, depending on the specific scenario). This structural convergence provides strong evidence that both models successfully identified genuine, useful clinical patterns to optimize patient outcomes within the simulated environment. Direct comparison with clinician decision-making was not possible because physician actions were not available in this study.

Treatment Intensity Analysis

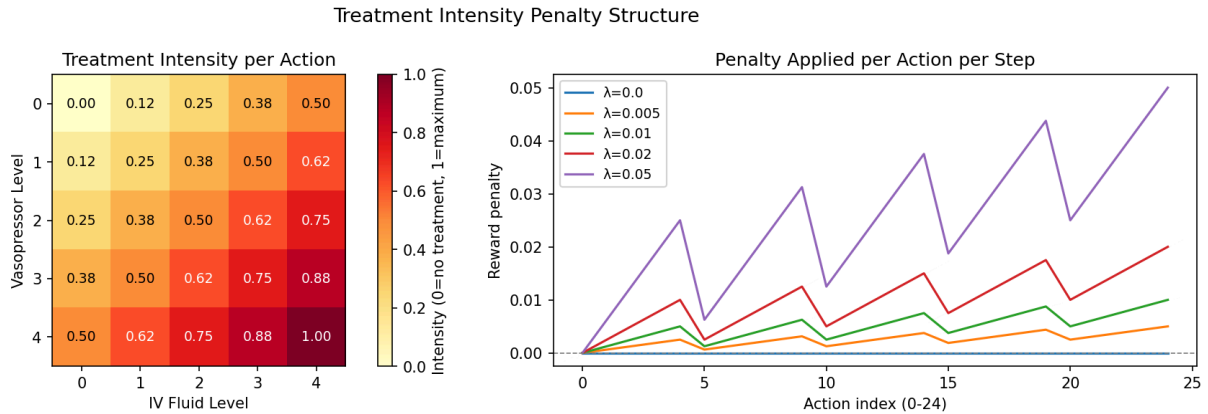


Figure 6: Treatment Strategy Distribution Analysis.

The treatment intensity penalty acts as a vital regulatory mechanism, controlled by the coefficient λ , which mathematically discourages aggressive medical interventions by directly modifying the reward function. As shown in the action matrix heatmap, the underlying intensity score scales linearly from 0.00 for complete inaction up to 1.00 for the simultaneous maximum administration of both vasopressors and IV fluids. When mapped across the 25 discrete action indices, this setup generates a distinct saw-tooth penalty pattern where the applied reward punishment per step scales proportionally with the chosen λ value. At our primary analytic baseline of $\lambda = 0.02$, executing a maximum-dosage combination inflicts an immediate per-step penalty of -0.02. This negative reinforcement directly mirrors high-stakes intensive care conditions, representing the clinical reality that delivering sustained maximum-tier medications introduces severe physiological toxicity and can kill a patient, thereby forcing reinforcement learning agents to actively penalize over-medication.

When comparing the two methods (see Fig B6, Appendix B), it is immediately clear that Dynamic Programming (DP) consistently maintains a much higher patient survival rate than SARSA across all settings. The final survival rate depends heavily on both the specific model used and the chosen lambda value. As a general rule, a higher lambda leads to lower survival results because the heavier penalization forces the models to back away from aggressive treatments to avoid penalties. For example, raising lambda from 0.0 to 0.05 drops DP's survival rate from ~85.7% to ~81.3% as it shifts to lower-dosage actions, while SARSA's performance remains trapped on a much lower plane near 74%. Ultimately, a penalization of $\lambda = 0.02$ proves to be a highly effective choice for our case, as the survival results at this specific threshold are either successfully maintained or even improved when compared to lower penalization values.

4. Conclusion

This project evaluated five reinforcement learning algorithms, Dynamic Programming (DP), Q-Learning, SARSA, DQN, and PPO, on the ICU-Sepsis-v2 benchmark across two observation-space configurations. The central finding is unambiguous: model-based planning is the only paradigm that reliably surpasses the random treatment baseline. DP achieved 83.2% survival (+8.9 pp over random) by exploiting direct access to the MDP transition model. All four model-free methods remained within ± 2 pp of their respective random baselines. The binding constraint is not algorithmic complexity but the sparse binary reward structure, with episodes averaging ~ 10 steps and a single terminal signal, learning from delayed rewards is unreliable at the training budgets used here.

In Configuration B, the irreducible mortality floor imposed by the ``AcuteEventEnv`` wrapper (see section 1.2) is the most challenging factor to learn by the agents. PPO achieves +1.8 pp over random, compared to DQN that consistently underperforms in comparison to the random policy, reflecting the advantage of on-policy training in matching the evaluation distribution. The interpretability analysis further shows that DP and PPO both learn vasopressor-focused strategies consistent with clinical practice, while SARSA collapses to near-total inaction ($\sim 74\%$ of steps) and Q-Learning produces no dominant strategy.

The creative extension introduced a per-step treatment intensity penalty (λ) and traced the resulting Pareto frontier between survival and intervention cost. This revealed that a meaningful reduction in treatment intensity can be achieved at low λ without a survival penalty, but that aggressive penalisation eventually conflicts with blood pressure stability, demonstrating that the binary survival objective alone is insufficient to capture clinically relevant treatment trade-offs.

Limitations and future work: Both DQN and PPO training curves did not converge at 100,000 steps. More fundamentally, dense reward shaping, for example, intermediate rewards tied to healthy vital sign ranges, would address the credit assignment problem that limits all model-free methods here. A model-based approach for Config B (e.g., a learned world model) could recover the planning advantage of DP without requiring exact transition probabilities. Finally, benchmarking trained policies against historical MIMIC-III clinician actions via off-policy evaluation would provide a clinically grounded reference beyond what survival rate alone can capture.

References

Johnson, A. E. W., Pollard, T. J., Shen, L., et al. (2016). MIMIC-III, a freely accessible critical care database. *Scientific Data*, 3, 160035.

Komorowski, M., Celi, L. A., Badawi, O., Gordon, A. C., & Faisal, A. A. (2018). The Artificial Intelligence Clinician learns optimal treatment strategies for sepsis in intensive care. *Nature Medicine*, 24, 1716–1720.

Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

Appendix

Appendix A: Hyperparameter Configurations

Table A.1: Optuna Hyperparameter Optimization Search Space for SARSA

Hyperparameter	Values Tested	Meaning & Clinical Function
Learning Rate (α)	[0.2, 0.1, 0.05]	How fast the agent learns: Controls how aggressively the agent updates its treatment beliefs based on new patient outcomes. Higher values make it shift strategies quickly; lower values make adjustments conservative.
Discount Factor (γ)	[0.9, 0.7, 0.5, 0.3, 0.1]	The agent's time horizon: Balances short-term goals versus long-term goals. High values force the agent to prioritize long-term recovery, while lower values force it to focus heavily on immediate patient stabilization.
Epsilon Start (ϵ_{start})	[0.9, 0.7, 0.5, 0.3]	Initial exploration rate: The starting probability that the agent will choose a random treatment combination to explore the simulation environment rather than exploiting its known best strategy.
Epsilon End (ϵ_{end})	[0.01, 0.1, 0.2, 0.3]	Final exploration floor: The minimum amount of random exploration left at the end of training. A low floor ensures that the finalized policy acts deterministically and safely chooses the best known treatment.
Epsilon Decay Steps	[3000, 2000, 1000, 500]	Exploration duration: The number of training episodes over which the exploration rate (ϵ) drops from its starting value to its final floor, dictating how long the agent stays in its trial-and-error discovery phase.
Learning Rate Decay (α_{decay})	['constant', 'linear']	Learning rate schedule: Determines if the learning speed remains fixed (constant) or gradually reduces over time (linear) to let the agent lock in stable, long-term clinical knowledge without changing its mind later.
Q-Table Initialization (q_{init})	[0.0, 1.0, 'random']	Initial expectations baseline: Sets the starting memory matrix values before training begins. Uniformly initializing at 0.0 prevents initial bias, ensuring the learned policies are derived entirely from empirical data without early optimism or pessimism.

Table A.2: DQN Final Hyperparameters

Hyperparameter	Value	Description
Learning Rate	10^{-4}	Step size for neural network weight updates
Batch size	64	Number of transitions sampled from the replay buffer per update
Buffer size	25 000	Maximum number of transitions stored in the replay buffer
Exploration fraction	0.25	Fraction of total training steps over which ϵ decays from 1.0
Exploration final (ϵ)	0.10	Minimum exploration rate after decay
Target update interval	500	Steps between target network synchronisations
Learning starts	2 000	Steps of random exploration before training begins
Train frequency	4	Environment steps between gradient updates
Gradient steps	1	Gradient updates per training step
Network architecture	[64, 64]	Two hidden layers of 64 units each
Activation function	ReLU	Non-linearity after each hidden layer (Kaiming uniform init)
Discount factor (γ)	1.0	No temporal discounting
Total timesteps	100 000	Total training budget

Table A.3: PPO Final Hyperparameters

Hyperparameter	Value	Description
Learning Rate	10^{-4}	Step size for policy and value network updates
n_steps	1 024	Steps collected per rollout before each update
Batch size	128	Mini-batch size for gradient updates
n_epochs	20	Gradient passes over each collected rollout
Clip range	0.3	PPO clipping parameter ϵ for the surrogate objective
Entropy coefficient	0.005	Entropy bonus weight to encourage exploration
GAE lambda	0.95	Generalised Advantage Estimation smoothing factor
Value function coefficient	0.5	Weight of the value loss in the combined objective
Max gradient norm	0.5	Gradient clipping threshold
Network architecture	[128, 128]	Two hidden layers of 128 units each
Activation function	ReLU	Non-linearity after each hidden layer (Orthogonal init)
Discount factor (γ)	1.0	No temporal discounting
Total timesteps	100 000	Total training budget

Appendix B: Supplementary Plots and Analysis

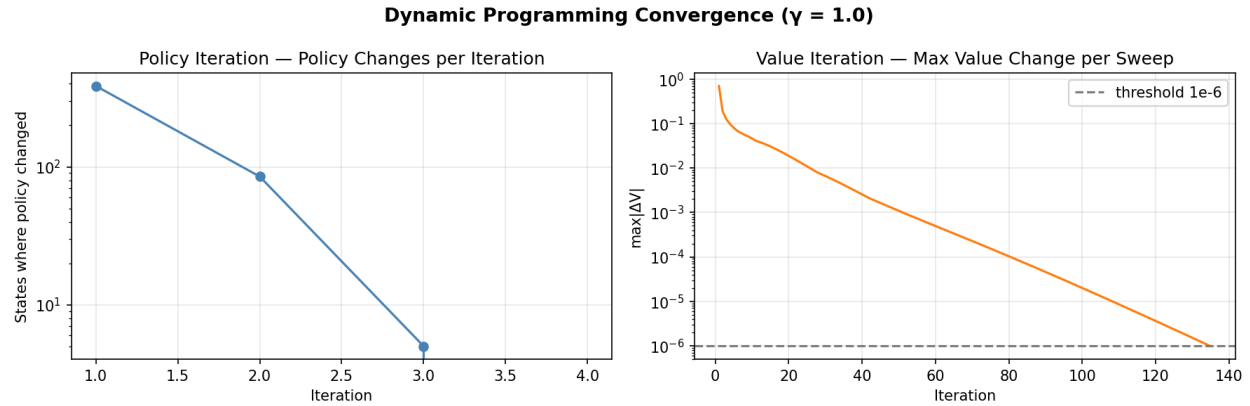


Figure B1: Dynamic Programming Convergence Profiles. Convergence comparison between model-based planning algorithms at $\gamma = 1.0$. Left: Policy Iteration. Right: Value Iteration.

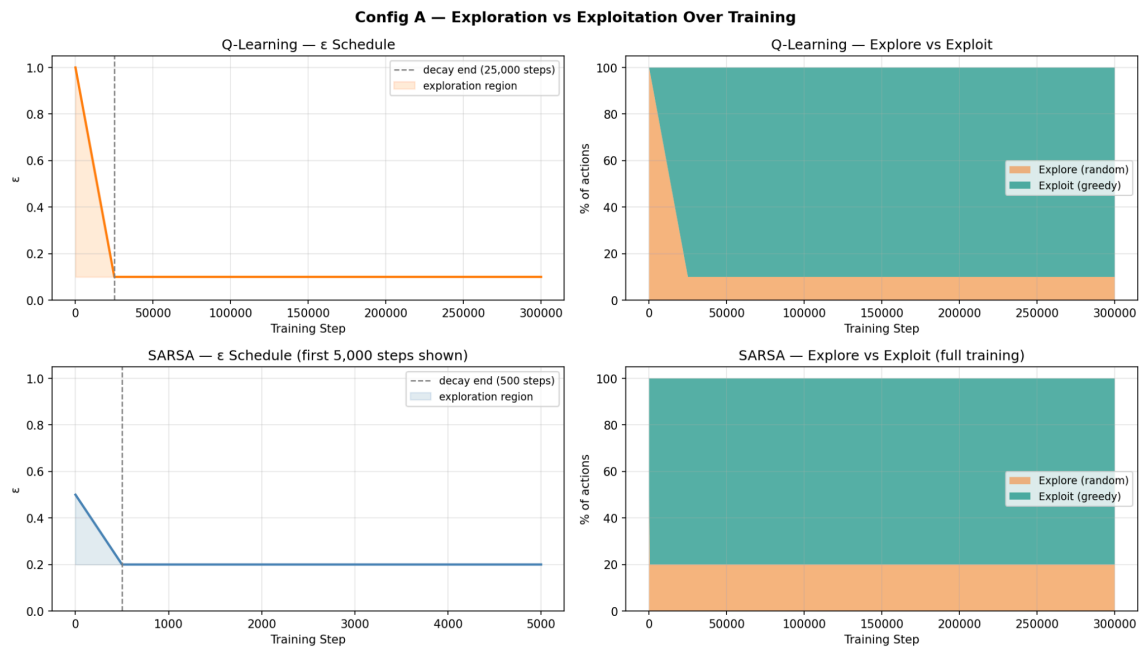


Figure B2: Tabular Agent Exploration-to-Exploitation Transitions (Config A, Q-Learning and SARSA).

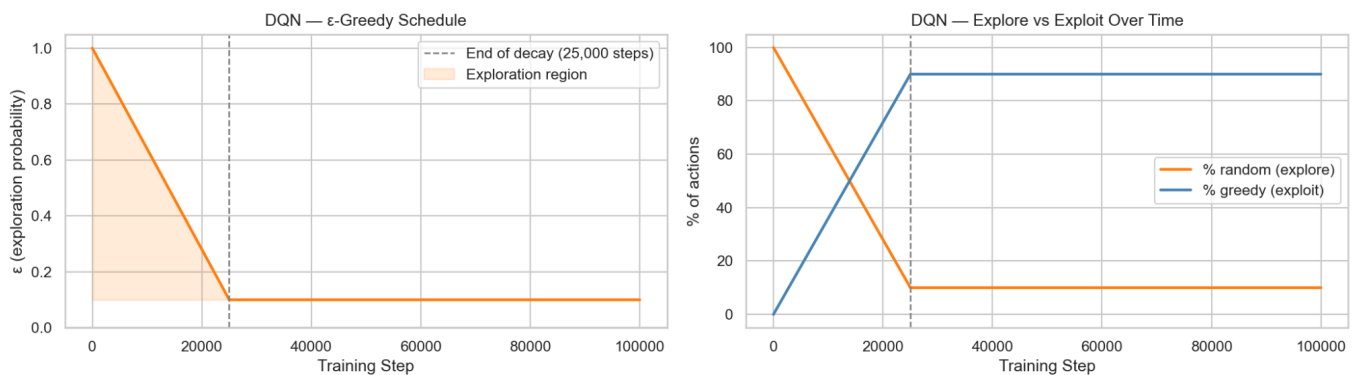


Figure B3: DQN exploration mechanics in Configuration B. The left panel shows the ϵ -greedy decay timeline transitioning over the first 25,000 timesteps. The right panel displays the empirical ratio of exploratory (random) actions versus greedy (exploitative) actions taken by the network over the total 100,000 training steps.

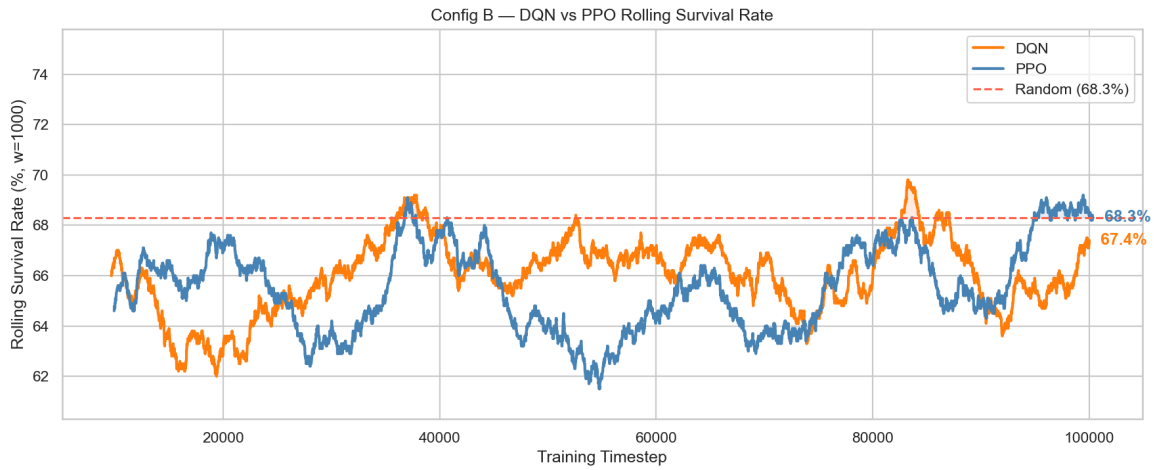


Figure B4: Rolling survival rate (window = 1,000 episodes) for DQN and PPO during training, with the random baseline shown as a dashed line.

6.2 Weight Initialization Comparison (DQN vs PPO, seed=42)

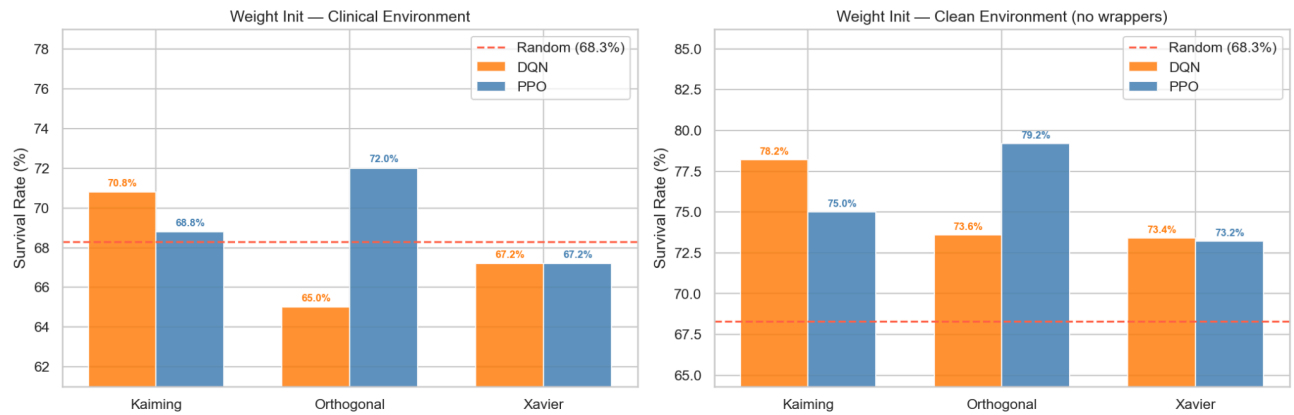


Figure B5: Weight initialization performance sensitivity comparison for DQN and PPO under seed 42, within both the full clinical environment (left) and the clean, wrapper-free environment (right).

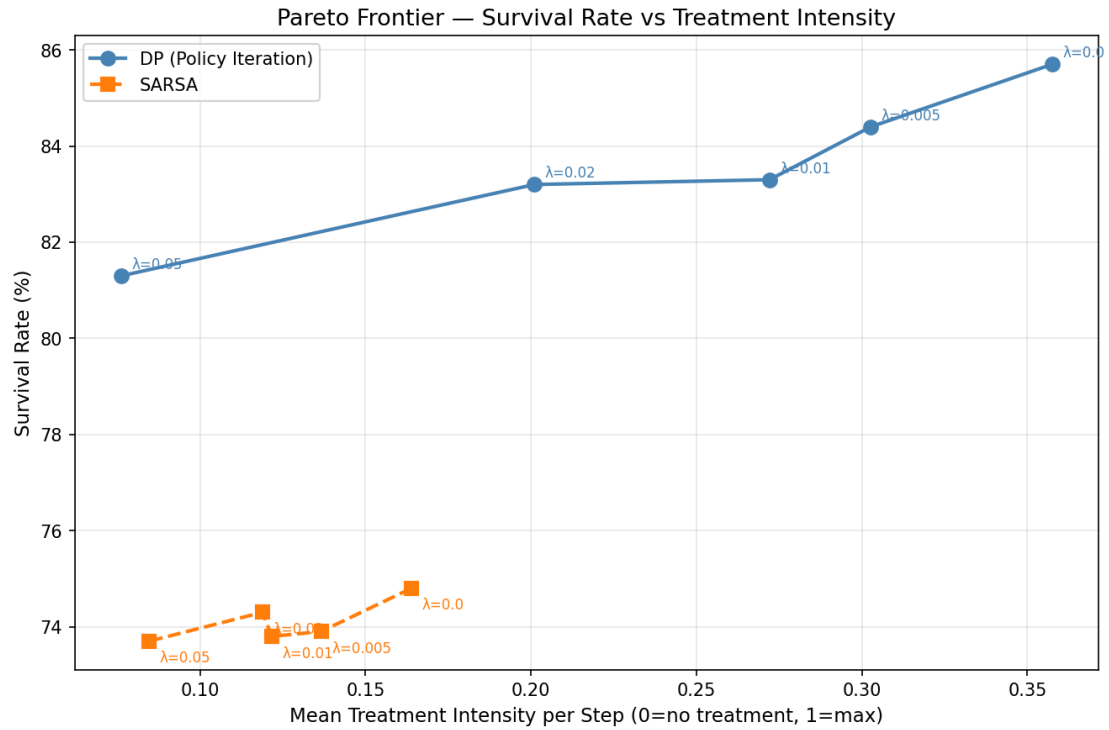


Figure B6: Survival Rate vs. Treatment Intensity in Configuration A. The curves contrast the true Pareto frontier achieved by model-based Dynamic Programming (Policy Iteration) (blue solid line) against the sub-optimal, sample-starved policies discovered by tabular SARSA (orange dashed line). The data demonstrates that moderate cost penalization ($\lambda = 0.02$) can significantly minimize treatment intensity without incurring a meaningful survival penalty.